

HW4: CUDA Matmul Optimization - Report

Team 23

R14922159 Lin, Hsin-Yu R14922112 Chen, Li-Jung R14945029 Lin, Wan-Hsuan

Optimizations applied, in the order you added them, with the measured GFLOPS after each step. A small table is enough.

Baseline and optimizations are shown in Table 1.

- **Baseline:** Naïve kernel (leveraging global-memory coalescing). Each thread computes one $C[i][j]$.
- **Optimization 1:** Shared memory tiling ($TILE_SIZE = 32$). Before computation, some A and B data are moved to shared memory.
- **Optimization 2:** 2D thread blocktiling + register caching + bank-conflict-free shared memory ($BM = BN = 128$, $BK = 16$, $TM = TN = 8$). Each thread computes a $TM \times TN$ C block. When loading A data into shared memory, it is transposed to make subsequent computation bank-conflict-free. Before computation, the data of A and B tiles in shared memory are loaded to the registers to accelerate subsequent computation.
- **Optimization 3:** Vectorized loads / stores + Warp-level tiling / autotuning + Double buffering. This utilizes `float4` to read and write 4 floats at a time. Double buffering allows iterations, excluding the first and last iterations, to simultaneously calculate tile i and load the data needed for tile $i+1$. The optimal combination of warp-level tiling / autotuning is $BM = BN = 128$, $BK = 16$, $TM = TN = 8$.

Table 1: GFLOPS with different optimizations applied across different sizes

Optimization \ Size	128	256	512	1024	2048	4096
Baseline: Naïve kernel (Global-memory coalescing)	324.099	1419.450	1594.241	1987.354	2158.255	2285.864
Opt. 1: Shared memory tiling ($TILE_SIZE=32$)	446.400	1996.223	2758.503	3478.682	3637.496	4143.701
Opt. 2: 2D Tiling + Reg caching + Bank-conflict-free	84.723	405.278	1799.003	6693.296	9200.188	11104.081
Opt. 3: Vec loads/stores + Warp-level + Double buffering	139.774	622.707	2635.855	10337.482	10808.322	13401.698

Final kernel design — key choices: tile sizes, thread tile, memory layout, vectorization, synchronization strategy. A short paragraph or a diagram.

- **Tile size:** 128×128
- **Thread tile:** 8×8
- **Memory layout:** We use a three-level hierarchy:
Global memory (A, B, C): row-major, no modification.
Shared memory (per block): Two double-buffered tiles, $sA[2][BK][BM]$ and $sB[2][BK][BN]$, which allow the thread to synchronously load the next set of data into another buffer while calculating the current data. The A tile is stored transposed — i.e. $sA[k][m]$ rather than

`sA[m][k]` — so that when each thread reads the `m`-strided column of `A` during the inner FMA loop, accesses stride contiguously across 32 shared-memory banks, avoiding bank conflicts. `B` is stored in a natural layout because its column access is already contiguous.

Registers (per thread): A $TM \times TN = 8 \times 8$ accumulator tile `C_tile[8][8]` (64 registers) plus two small register caches `regA[8]`, `regB[8]` reloaded each `k` step, and two staging buffers `ldgA`, `ldgB` (`float4` arrays) holding the next tile prefetched from global memory while the current tile is being computed.

- **Vectorization:** Use `float4` for 128-bit access. When reading matrices `A` and `B` from global memory and writing back matrix `C`, use `float4` so that a single 128-bit instruction can read/write 4 floats at a time, making more efficient use of the memory bandwidth.
- **Synchronization strategy:** Double Buffering is used in conjunction with `__syncthreads()`. Before proceeding to the next iteration, `__syncthreads()` is invoked to ensure that all threads have completed writing the data from `ldgA/B` (register) to `sA/sB` (shared memory).

What you tried that didn't work, if anything. Brief.

Initially, based on intuition, matrices `A` were loaded directly into shared memory (`sA[row][col]`) in its original row-major format. Without transposition, threads within the same warp will simultaneously access different addresses in the same shared memory bank, causing severe bank conflict and resulting in memory reads being serialized, significantly slowing down the throughput of FMA operations.

(Optional) A scaling plot of GFLOPS vs. matrix size.

As shown in Figure 1, when the matrix size reached 1024, a distinct crossover occurred. Optimization 3 demonstrated excellent scalability, with performance showing explosive growth, becoming the optimal version and peaking at 13,402 GFLOPS at a size of 4096, which demonstrates the vectorized load/store and double buffering can further improve the GFLOPS.

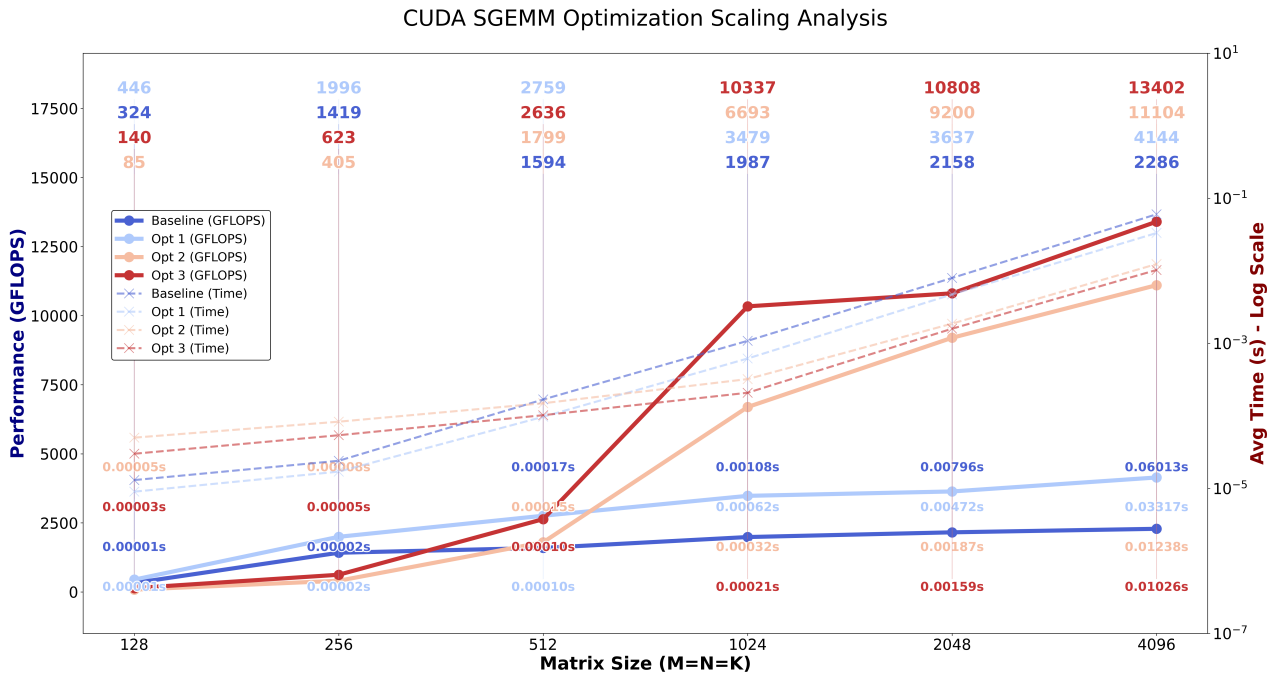


Figure 1: GFLOPS avg. time vs. matrix size.